

DATA STRUCTURES AND ALGORITHM-FinancialForecasting

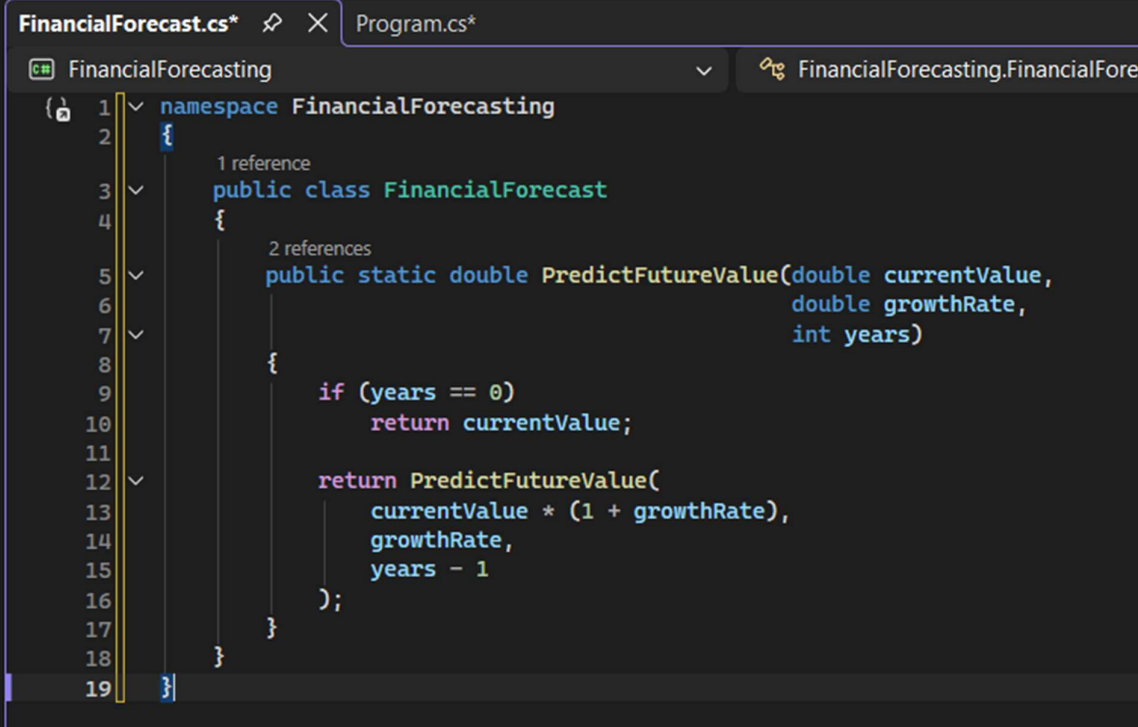
Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. **Understand Recursive Algorithms:**
 - Explain the concept of recursion and how it can simplify certain problems.
2. **Setup:**
 - Create a method to calculate the future value using a recursive approach.
3. **Implementation:**
 - Implement a recursive algorithm to predict future values based on past growth rates.
4. **Analysis:**
 - Discuss the time complexity of your recursive algorithm.
 - Explain how to optimize the recursive solution to avoid excessive computation.



The screenshot shows a code editor with two tabs: 'FinancialForecast.cs*' and 'Program.cs*'. The 'FinancialForecast.cs*' tab is active, displaying the following C# code:

```
1 namespace FinancialForecasting
2 {
3     1 reference
4     public class FinancialForecast
5     {
6         2 references
7         public static double PredictFutureValue(double currentValue,
8             double growthRate,
9             int years)
10        {
11            if (years == 0)
12                return currentValue;
13
14            return PredictFutureValue(
15                currentValue * (1 + growthRate),
16                growthRate,
17                years - 1
18            );
19        }
20    }
```

```
FinancialForecasting FinancialForecasting.Program
1 namespace FinancialForecasting
2 {
3     0 references
4     internal class Program
5     {
6         0 references
7         static void Main(string[] args)
8         {
9             double currentValue = 10000;
10            double growthRate = 0.10; // 10%
11            int years = 5;
12
13            double futureValue =
14                FinancialForecast.PredictFutureValue(
15                    currentValue,
16                    growthRate,
17                    years
18                );
19
20            Console.WriteLine($"Current Value : {currentValue}");
21            Console.WriteLine($"Growth Rate : {growthRate * 100}%");
22            Console.WriteLine($"Years : {years}");
23            Console.WriteLine($"Future Value : {futureValue:F2}");
24        }
25    }
```

OUTPUT

```
Microsoft Visual Studio Debug Console
Current Value : 10000
Growth Rate : 10%
Years : 5
Future Value : 16105.10

D:\DN5-DotNet-FSE-1\Week-1\Data-Structures-Analysis\FinancialForecasting.exe (process 6668) exited with code 0.
To automatically close the console when debugging stops, enable the option 'Automatically close console when debugging stops'.
Press any key to close this window . . .
```

1. What is Recursion?

Recursion is a programming technique where a method calls itself to solve smaller instances of the same problem. It simplifies problems that can be broken into repetitive subproblems.

2. Recursive Formula Used

$$\text{FutureValue}(n) = \text{FutureValue}(n-1) \times (1 + \text{GrowthRate})$$

Base Case:

$$\text{Years} = 0$$

3. Time Complexity

For this recursive solution:

$$T(n) = T(n-1) + O(1)$$

Therefore:

$$\text{Time Complexity} = O(n)$$

where n is the number of years.

Space Complexity: $O(n)$

because each recursive call is stored on the call stack.

4. Optimization

The recursive solution can be optimized by:

1. Using an iterative loop.
2. Using the compound interest formula:

$$\text{Future Value} = \text{Present Value} \times (1 + \text{Growth Rate})^{\text{Years}}$$